

Typed, Calibrated Decisions as a Reward Source *and* as a Critic for Reinforcement Learning Fine-Tuning of Language Models

Abstract

Reinforcement learning fine-tuning of language models needs a source of reward, and every source in common use is a compromise. A Bradley–Terry reward model is cheap but frozen in the preference distribution it was fit to and capped by a short context window. An LLM judge is flexible but expensive, slow, and read through a lossy text-parsing step. A self-judge is free but, at small scale, close to noise. We study a fourth option that has not previously been used for RL: a *System One* model, which emits typed, calibrated answers to predeclared questions and never generates text. We use JEV (TypeSafe) in two roles. As a **reward function** it answers a fixed rubric about a finished response in one request, returning per-question probabilities directly, with no parsing. As a **critic** it answers a forward-looking question about a *partial* response — “will this, once finished, be correct and helpful?” — which is exactly the quantity PPO’s value network is trained to predict. We show empirically that this prefix probability tracks eventual response quality (AUC = 0.84 at the full response and = 0.77 after only a quarter of it; $R^2 = 0.52$), which licenses replacing PPO’s learned value head with a frozen, zero-shot, externally calibrated critic: no value parameters, no value loss, no bootstrapping from a critic that is itself still learning. We compare five reward sources under GRPO at a matched reward-call budget, and two critics under PPO, holding the start checkpoint, prompt set, prompt order, decoding parameters and KL coefficient fixed, and evaluating at matched KL rather than at matched step count. We report a cross-reward matrix that separates genuine improvement from over-optimisation of whichever judge a run was trained against, and full cost accounting. [\[headline results once runs complete\]](#)

1 Introduction

The standard recipe for aligning a language model with reinforcement learning fixes an optimiser — PPO [Schulman et al., 2017, Ouyang et al., 2022] or a critic-free variant such as GRPO [Shao et al., 2024] — and varies almost everything else. In practice the component that determines whether the run is useful, and most of what it costs, is the *reward source*: the thing that turns a sampled response into a number.

Three families dominate. A **learned reward model** fits a Bradley–Terry objective to human preference pairs [Bradley and Terry, 1952, Stiennon et al., 2020] and is then frozen; it is nearly free to query, but inherits the distribution of the preference data and, for the encoder models typically used, a 512-token window. **LLM-as-judge** / RLAIFF prompts a capable instruct model to grade the response [Bai et al., 2022, Lee et al., 2024, Zheng et al., 2023]; it generalises far better, but each judgment is an autoregressive generation that must be parsed, and the parse is a lossy, brittle step between the judge’s belief and the scalar the optimiser sees. **Self-rewarding** setups [Yuan et al., 2024] let the policy grade itself, which removes the external cost but, at the model scales most practitioners can afford to train, produces a judge barely better than chance.

A fourth kind of model has recently become available and does not fit this taxonomy. TypeSafe’s JEV [TypeSafe AI, 2026] is a *System One* model: it is given a state and a set of questions declared in advance, and it returns *typed* answers — a probability for a yes/no proposition, a distribution over named options, an expected level on an ordered scale — together with calibration information. It does not produce text. Because the answer space is fixed by the request, the model cannot return a value outside the schema, and because no tokens are decoded, a judgment costs roughly one prefill and returns in a fraction of a second.

For reinforcement learning this combination is unusually well matched to the problem, in two separate ways, and the distinction between them is the subject of this paper.

As a reward function. A rubric is a set of predeclared questions. That is precisely JEV’s input format, so a multi-criterion reward becomes one request, the per-criterion answers come back as numbers rather than as prose to be parsed, and the scalarisation is the experimenter’s choice rather than an artefact of how a judge happened to phrase itself.

As a critic. In a language-model MDP with a single terminal reward R , the value function is

$$V^\pi(s_t) = \mathbb{E}[R | x, y_{<t}], \quad (1)$$

the expected quality of the finished response given the prefix generated so far. This is a question about a partial state with a yes/no answer, which is exactly the shape JEV accepts. PPO normally learns (1) with a value head trained online against its own returns. If an external model can answer it zero-shot, in calibrated units, for a fraction of a cent, the value network becomes unnecessary — and with it the value parameters, the value loss, the value optimiser state, and the bootstrapping bias of a critic that is itself still being fit. To our knowledge no prior work uses an external judge as a token-level critic; judges are used to produce terminal rewards.

A design note on what we did *not* test. A natural fourth condition — “JEV as an evaluator” — collapses into the reward-function condition. Scoring a finished response and taking the resulting scalar as the learning signal *is* using JEV as a reward function; running it as a separate arm would duplicate a column. We therefore drop it and spend the comparison budget on the critic role, which is genuinely distinct because it queries states the reward function never sees.

Contributions.

1. The first study of a typed-decision (System One) model as a reward source for RL fine-tuning, compared against a Bradley–Terry reward model, an LLM judge (RLAIF) and a self-judge under an identical rubric, identical scalarisation, and a matched budget of reward calls.
2. JEV as a *critic*: we show its prefix probability is an empirically calibrated estimator of final response quality, and use it to run PPO with no value network at all.
3. A fairness protocol that reads the LLM and self-judges *from their logits* rather than by parsing text, so that the dense, parse-free property is not silently granted to JEV alone.
4. An ablation on where the rubric comes from (STRUCTUREDGENERATION2JEV): fixed, generated per prompt by a frozen LLM, or proposed by the policy being trained — the last as a deliberate reward-hacking probe.
5. Cost and latency accounting for every arm, measured rather than quoted.

Table 1: Measured properties of JEV 1.13 accessed through OpenRouter’s decisions endpoint. Latency and cost are measured from our own calls, not quoted from documentation; they include network round-trip from the client region and so upper-bound the provider-side figures.

Property	Value
Model	<code>typesafe/jev-1.13</code>
Access	OpenRouter <code>/api/alpha/decisions</code>
Question types	NOUL, CHOICE, SCORE
Output	typed values with probabilities; no text
Context	32,000 tokens
Measured cost	\$0.0299 per 1,000 calls (5-question rubric)
Measured latency p50 / p95	0.37 s / 0.99 s
Measured throughput	25.4 calls/s at concurrency 32
Mean input tokens per call	713

2 Background

2.1 RL fine-tuning

We use two optimisers. **PPO** [Schulman et al., 2017] maximises a clipped surrogate with advantages from GAE [Schulman et al., 2016], requiring a critic V . **GRPO** [Shao et al., 2024] removes the critic by sampling G completions per prompt and using the group mean as the baseline; it is the natural setting in which to vary the reward source alone, since no value network confounds the comparison.

2.2 Reward sources

See Section 3 for exact definitions. Briefly: a frozen DeBERTa Bradley–Terry model; a frozen instruct LLM read through logits; a frozen copy of the SFT checkpoint read the same way; JEV; and, as a degenerate control, the policy’s own token-level self-certainty, which consults nothing external.

2.3 System One models and Jev

JEV accepts a *state* and a map of *questions*, and returns one typed answer each [TypeSafe AI, 2026]. Three question types exist: a NOUL returns $P(\text{true}) \in [0, 1]$ for a proposition; a CHOICE returns a distribution over named options plus a confidence; a SCORE returns an expected level on an ordered rubric of 2–10 levels, plus the level distribution and a confidence. All questions in a request are answered against the same state, and adding questions has little effect on latency. We access JEV 1.13 through OpenRouter’s decisions endpoint. Measured properties relevant to RL are given in Table 1.

3 Method

3.1 A single rubric, rendered into each judge’s native format

Every judge-based arm answers the same five questions about a (request, response) pair: CORRECT (noul, +), HELPFUL (3-level score, +), PADDED (noul, −), EVASIVE (noul, −) and SELF_PROMO

(noul, -). Writing $\hat{q}_i \in [0, 1]$ for the normalised answer to question i (a score of L levels is divided by $L - 1$), the reward is

$$R = \hat{q}_{\text{correct}} + \hat{q}_{\text{helpful}} - \hat{q}_{\text{padded}} - \hat{q}_{\text{evasive}} - \hat{q}_{\text{self_promo}} \in [-3, +2]. \quad (2)$$

SELF_PROMO is not optional. The policy’s own text sits inside the judge’s state, so a policy can learn to assert its own quality at the judge; we both penalise it in the reward and track it as a hacking detector (Section 4.1).

3.2 Reading judges without parsing

Comparing a typed model against a text-generating judge invites an unfair advantage: if the LLM judge is read by parsing its prose, we are partly measuring parse failures. We therefore read every generative judge from its logits. For a noul we take $P(\text{yes})/(P(\text{yes}) + P(\text{no}))$, summing over capitalisation and leading-space variants of each token; for a score we take a softmax over the level-index tokens and its expectation. Each question is a separate prefill sharing the state prefix, mirroring how JEV evaluates questions in parallel. The scalarisation (2) is then byte-identical across JEV, the RLAIF judge and the self-judge.

This applies to the *hosted* RLAIF judge as well, which matters because the judges small enough to run on our hardware sit at or near chance on held-out preference pairs (Section 5.1); comparing against one of those would be comparing against a straw man. Three practical details make a hosted judge readable:

- **Reasoning must be off.** A reasoning model spends its first tokens on thought and returns no content to read; with reasoning disabled the answer token is the first token.
- **The provider must be pinned.** The gateway load-balances across providers and only some return `logprobs`; without requiring the parameter, a fraction of calls silently fall back to parsing. Requiring it took the unreadable fraction from 50% to under 5%.
- **Prompt order is state-then-question.** The rubric’s questions about one completion share a long prefix (system and state) and differ only in a short suffix, so the state goes first. This is the reverse of the usual “instructions first” caching advice, which optimises the wrong axis when the repeated span is the state rather than the instruction.

Calls whose logprobs are still unreadable fall back to the emitted word, and we report that fraction rather than leaving it implicit.

3.3 Jev as a reward function

One request carries the state and all five rubric questions; the answers are normalised and combined by (2). No parsing, no retry-on-malformed, and one network round trip per completion regardless of rubric size.

3.4 Jev as a critic

We ask JEV a single forward-looking noul about a *partial* response: *if the assistant continues in the same direction and finishes, will the completed response be a correct and genuinely helpful answer?* Write p_t for the returned probability at prefix length t .

Algorithm 1 PPO with JEV as the critic. Differences from textbook PPO are the lines marked $\triangleright$; everything else, including the clipped surrogate and the KL shaping, is unchanged.

```

1: calibrate once: sample  $(x, y)$  from  $\pi_{\text{SFT}}$ ; fit  $a, b$  minimising  $\sum (a p(x, y) + b - R(x, y))^2$   $\triangleright \triangleright$ 
   puts  $\hat{V}$  in reward units
2: for each step do
3:   sample completions  $y \sim \pi_\theta(\cdot | x)$ 
4:    $R \leftarrow \text{rubric}(x, y)$   $\triangleright$  terminal reward, one JEV call per completion
5:    $r_t \leftarrow -\beta(\log \pi_\theta(y_t) - \log \pi_{\text{ref}}(y_t))$ ;  $r_{T-1} += R$ 
6:   for  $k = 1 \dots K$  do  $\triangleright \triangleright K$  prefix probes, not a value net
7:      $t_k \leftarrow \lfloor k(T-1)/(K-1) \rfloor$ ;  $p_{t_k} \leftarrow \text{JEV}(x, y_{<t_k}, q_{\text{prefix}})$ 
8:   end for
9:    $\hat{V}(s_t) \leftarrow a \cdot \text{INTERP}(t; \{t_k\}, \{p_{t_k}\}) + b$ ,  $\hat{V}(s_T) \leftarrow 0$   $\triangleright \triangleright$  frozen, no gradient
10:   $\delta_t \leftarrow r_t + \gamma \hat{V}(s_{t+1}) - \hat{V}(s_t)$ ;  $A_t \leftarrow \sum_l (\gamma \lambda)^l \delta_{t+l}$ 
11:  update  $\theta$  on the clipped surrogate  $\triangleright \triangleright$  no value loss term
12: end for

```

Calibration. p_t lives in $[0, 1]$ and R in $[-3, +2]$, so TD errors computed from raw p_t would be in arbitrary units. We fit $\hat{V}(s_t) = a p_t + b$ by least squares on held-out (complete response, reward) pairs drawn from the SFT policy, and report R^2 , so the quality of the map is visible rather than assumed (Section 5.2).

Sparse probing. Querying every token position is unnecessary and would dominate cost. We probe K positions per completion, evenly spaced in token index, and interpolate $\hat{V}$ piecewise-linearly between them; K is an ablation axis. Advantages then follow standard GAE with $\gamma = 1$, using $\hat{V}(s_T) = 0$ at the terminal state.

What this removes. The Jev-critic arm has no value head, no value loss term, no value learning rate and no value optimiser state. Against the learned-critic arm it differs in exactly that, since both run through the same PPO implementation.

3.5 What sparse probing costs in principle

Probing K positions and interpolating is an approximation, and it is worth saying exactly what it approximates and how badly, because that is what decides how to choose K and what the resulting advantages mean.

Proposition 1 (Credit is assigned per segment, not per token). *Let $\hat{V}$ be affine on each interval between consecutive probes t_k, t_{k+1} , and let $\gamma = 1$. Then for $t_k \leq t < t_{k+1} - 1$ the temporal-difference error is*

$$\delta_t = r_t + \hat{V}(s_{t+1}) - \hat{V}(s_t) = r_t + \frac{p_{t_{k+1}} - p_{t_k}}{t_{k+1} - t_k} a,$$

which is constant in t within the segment apart from the per-token shaping term r_t . Consequently the critic distinguishes credit only at the granularity of the $K - 1$ segments.

This is a structural difference from a learned value head rather than an artefact of insufficient probing: no increase in per-probe accuracy makes the interpolated critic discriminate between two tokens inside one segment. It is also why K is an experimental axis rather than an implementation detail.

Proposition 2 (Bias of the interpolated critic). *Write $V^*(t)$ for the true value at token index t and $h = T/(K - 1)$ for the probe spacing. If V^* is L -Lipschitz in t , then*

$$\sup_t |\hat{V}_K(t) - V^*(t)| \leq \frac{1}{2}Lh = O(T/K).$$

If in addition the second difference of V^ is bounded by M , the bound tightens to $\frac{1}{8}Mh^2 = O(T^2/K^2)$.*

Both are the standard linear-interpolation bounds; what is not standard is which regime a real value curve occupies, and that is an empirical question we answer in Section 5.3 by probing every position on a small sample and measuring how the error actually decays with K .

Corollary 1 (The bias reaches the optimiser undiminished). *With $\gamma = \lambda = 1$ the generalised advantage telescopes to $A_t = R - \hat{V}(s_t)$, so the advantage error equals the value error pointwise and inherits the bounds of Proposition 2.*

Cost of the critic. The arm spends K extra JEV calls per completion. At the measured price this is a fraction of a cent per step and buys token-level credit assignment; the comparison in Section 5.2 is what decides whether the signal is worth it, and Table 11 reports what it cost.

3.6 StructuredGeneration2Jev

The component that decides *which* questions JEV is asked is a swappable axis with three settings. **static:** the fixed rubric of Section 3.1; the only setting under which JEV, the LLM judge and the self-judge are comparable, and therefore the setting used in the main pipeline. **generated:** a frozen LLM writes up to three task-specific questions per *prompt*, each carrying an explicit sign, appended to the fixed rubric; questions are generated per prompt and cached, never per completion, since a reward that depends on the completion through the question set is not a function of the completion and makes the RL objective ill-posed. **policy:** the model being trained proposes its own questions — a deliberate reward-hacking probe, since nothing prevents it from proposing criteria it trivially satisfies. Generated questions are schema-validated and dropped if malformed; rewards are rescaled by the rubric’s attainable range so that a longer rubric does not inflate the reward scale.

3.7 Controlled variables

Identical across every run: the SFT start checkpoint, the prompt set and its order (fixed dataloader seed, verified by a logged fingerprint), decoding parameters, KL coefficient, G for GRPO, and the *budget of reward calls* rather than the number of steps. We use three seeds per arm and report mean $\pm$ standard deviation.

4 Experimental setup

The policy is a small Qwen2.5 instruct model [Qwen Team, 2025]; prompts and preference pairs come from UltraFeedback [Cui et al., 2023]; training uses TRL [von Werra et al., 2020] for SFT, GRPO and the reward model, and our own PPO implementation for the critic comparison, because TRL 1.13 no longer ships a PPO trainer and because one implementation with a swappable critic is what makes that comparison exact.

Table 2: Configuration. Every entry is held fixed across all arms except the reward source (GRPO phase) and the critic (PPO phase).

Component	Setting
Policy	Qwen/Qwen2.5-0.5B-Instruct
SFT data	UltraFeedback (binarized), chosen responses
RL prompts	UltraFeedback <code>train_prefs</code> , fixed order
Held-out prompts	UltraFeedback <code>test_prefs</code> , 200
Ground-truth task	GSM8K, 250 test problems
BERT RM	OpenAssistant/reward-model-deberta-v3-large-v2
RLAIF judge	Qwen/Qwen2.5-3B-Instruct, read from logits
Self-judge	frozen copy of the SFT checkpoint
JEV	typesafe/jev-1.13
Evaluation judge	openai/gpt-5-mini (unrelated to every training judge)
GRPO	$G = 8$, 16 prompts/step, 180 steps, lr 10^{-6} , $\beta = 0.04$
PPO	64 prompts/step, 110 steps, lr 10^{-6} , KL coef 0.05, GAE $\lambda = 0.95$
Reward-call budget	23,040 per GRPO run (matched across sources)
Decoding	temperature 1.0, top- p 1.0, max 384 new tokens
Seeds	3 per core arm
Hardware	2×NVIDIA T4 (16 GB), Kaggle

Why the policy is adapted rather than fully fine-tuned. On the 14.6 GB card available, full fine-tuning of even a 0.5B policy does not fit at *any* micro-batch size, and the reason is worth stating because it is not the one usually assumed. The weights are 1.8 GB; the gradients are another 1.8 GB, the optimiser moments 3.7 GB, the separate reference policy 1.8 GB and the rollout KV cache 2.6 GB — together over the card before a single logit is allocated, and the logits for a 150k vocabulary are themselves 4 GB at a micro-batch of eight. LoRA removes three of those terms at once, since the reference policy becomes the base model with adapters disabled. Every arm uses an identical adapter configuration, so the comparison between reward sources is unaffected; absolute quality under full fine-tuning would differ, and we do not claim otherwise. Our memory model (`scripts/memory_budget.py`) reproduces the observed failure to within 0.5 % and is included so the calculation can be redone for other hardware.

4.1 Evaluation

Quality. Head-to-head win-rate against the SFT checkpoint on held-out prompts, judged by a model from a family used by *no* training reward source. Each pair is judged twice with positions swapped and averaged, removing position bias. The headline number is length-controlled in the AlpacaEval-2 sense [Dubois et al., 2024]: we fit $P(\text{win}) = \sigma(\theta + \gamma \Delta \ell)$ over standardised length differences and report $\sigma(\theta)$. Confidence intervals are bootstrap over prompts.

Matched divergence. Arms are compared at equal $\text{KL}(\pi \parallel \pi_{\text{ref}})$, not at equal step count; a reward source that moves faster per step is not thereby better.

Hacking detectors. GSM8K accuracy [Cobbe et al., 2021] (a judge can be talked into approval; arithmetic cannot), mean response length, self-praise phrase rate, distinct-2, and the gap between training reward and held-out quality.

Table 3: How well each reward source recognises a human preference, measured before any RL. Each source scores the chosen and the rejected response of $n = 1400$ held-out UltraFeedback pairs; accuracy is how often it ranks the chosen one higher, and Cohen’s d is the separation in units of the margin’s own standard deviation. A source that cannot do this will not teach a policy anything, so this is what fixes the RLAIIF judge size rather than an assumption. Wall-clock is for all $2n$ scorings on the same hardware.

Reward source	Accuracy (%)	Cohen’s d	Mean margin	Wall-clock (s)
api_judge:deepseek-v4-flash-0731	68.0	+0.53	+0.573	539
JEV	66.3	+0.43	+0.487	19
LLM judge Qwen2.5-7B-Instruct	65.1	+0.40	+0.506	8704
LLM judge Qwen2.5-3B-Instruct	62.3	+0.32	+0.239	3767
BERT RM (DeBERTa BT)	58.8	+0.22	+0.399	162

Cross-reward matrix. Over-optimising a learned proxy is a well-documented failure of RLHF [Gao et al., 2023], and a judge-based reward is a proxy like any other. Every final checkpoint is scored by *every* reward source. The diagonal is what a run optimised; the off-diagonal is what the other judges make of it. We report $\text{diag} - \text{off-diag}$ in baseline-relative units as an over-optimisation index.

Cost. Dollars, wall-clock, reward-call count and reward latency (p50/p95), measured in-run rather than quoted from documentation.

5 Results

5.1 Can each reward source recognise a human preference at all?

A comparison is only informative if every arm’s judge works. Table 3 measures each source on held-out preference pairs before any RL, which is what fixes the RLAIIF judge size; running that arm with a judge at chance would be a straw man rather than a baseline.

Two things in that table matter more than the ranking. First, the local judges that fit on the available hardware are weak: the 1.5B model is at chance, and the 3B model is below the Bradley–Terry reward model it is supposed to improve on. That is why the RLAIIF arm uses a hosted judge, read from logits so that the fairness protocol of Section 3.2 still holds.

Second, the efficiency claim, stated carefully because the obvious version of it is not defensible. Scoring the 2,800 judgements of Table 3 took 19s through JEV and 8704s with **Qwen2.5-7B-Instruct**, at accuracies of 0.663 and 0.651. That ratio compares two *deployment modes*, not two models, so we swept both sides rather than quoting it: JEV across concurrency and the local judge across batch size (Table 4).

The sweep did not go as expected. The local judge’s throughput *falls* monotonically as its batch grows — 0.72, 0.61 and 0.51 judgements per second at batches of 4, 8 and 16 — while its memory rises from 4.3 to 8.5 GB, and a batch of 32 runs out of memory. The cause is structural rather than incidental: a rubric of Q questions is Q separate prefills over a shared state, so a larger batch mostly buys padding. Batching is the obvious lever and it points the wrong way. We therefore compare each side at its own best setting, which is the comparison that favours the local judge as far as it can be favoured.

What does not depend on tuning is where the computation has to happen. During RL the local

Table 4: Judgements per second against the setting that controls them, on $n = 200$ pairs. Sweeping the local judge’s batch size separates the part of its disadvantage that tuning closes from the part that does not: the peak-memory column is taken from the policy during RL, and no batch size returns it.

Configuration	Judgements/s	Peak VRAM (GB)	USD
local, batch 4	0.72	4.3	–
local, batch 8	0.61	5.7	–
local, batch 16	0.51	8.4	–
JEV, concurrency 8	33.64	0.0	0.0089
JEV, concurrency 16	56.92	0.0	0.0089
JEV, concurrency 32	83.54	0.0	0.0089
JEV, concurrency 48	91.19	0.0	0.0089

Table 5: JEV’s forward-looking probability on a *partial* response, as a function of how much of the response it has seen, measured on $n = 487$ completions sampled from the SFT policy. “Good” and “bad” are the above- and below-median halves by *final* rubric reward, which is not observable at the time of the query. AUC is the probability that the prefix value ranks an eventually-good response above an eventually-bad one; R^2 is for the affine fit of final reward on prefix value.

Prefix seen	$\bar{p}$ (good)	$\bar{p}$ (bad)	AUC	Spearman ρ	R^2
0%	0.530	0.508	0.573	0.185	0.032
10%	0.622	0.460	0.686	0.364	0.129
25%	0.581	0.325	0.774	0.521	0.237
50%	0.520	0.215	0.838	0.642	0.361
75%	0.478	0.168	0.855	0.688	0.422
100%	0.449	0.139	0.841	0.687	0.515

judge occupies the same accelerator as the policy it is scoring: it cannot overlap with generation, and on the 14.6 GB cards used here a 7–8B judge does not fit beside the policy at all — it had to be sharded across both GPUs simply to run. A judge reached over the network contends for nothing. This matters more in the critic role than in the reward role, because a critic is queried K times per completion rather than once, so the arithmetic that decides feasibility is K times harsher. We therefore report wall-clock, peak memory and dollars per judgement and let the reader apply their own hardware, rather than resting the argument on a single ratio.

5.2 Is Jev’s prefix probability a value function?

The mechanism transfers, and the reading we expected does not. All three readers answer the prefix question usefully: AUC against the eventual outcome rises with the fraction of the response seen for each of them, so the critic role is a property of *readable calibrated belief* rather than of one vendor’s model. That is the claim we set out to test, and it holds.

What does not hold is the reading we expected of the second panel. We had supposed that a saturated critic would be a worse critic, because a temporal-difference error is a difference between neighbouring values and a curve pinned at 0 or 1 has none to give. The data do not support that as stated: the hosted judge saturates on three quarters of responses and yet holds the *widest* separation and the highest terminal AUC. Saturation and discrimination are not in conflict — pushing values to the extremes is what widens the gap between groups, at the cost of resolution

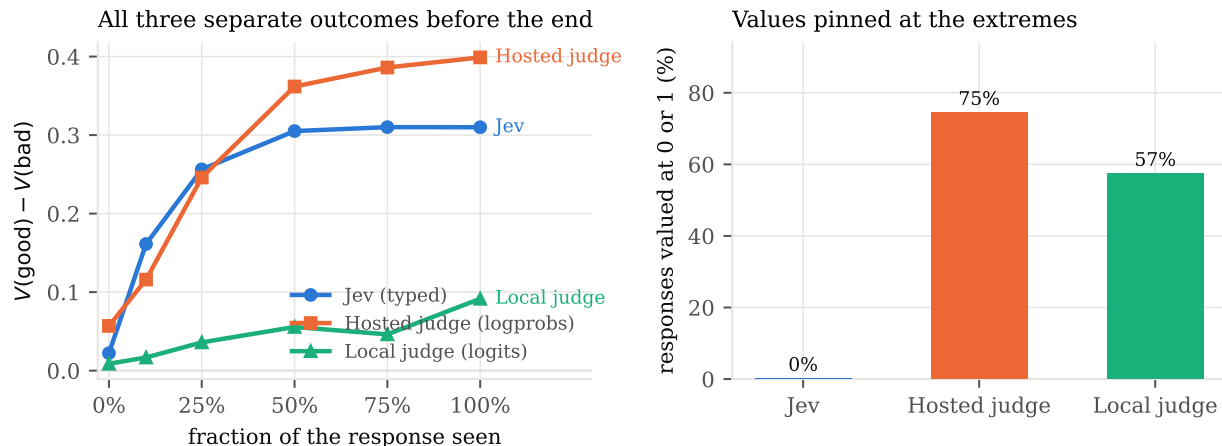


Figure 1: Left: the gap each critic holds between prefixes that end well and prefixes that end badly, as a function of how much of the response it has seen. The outcome is not observable at the time of the query. All three separate the two groups before the response ends, which is the claim the critic role rests on, and the hosted judge separates them most. Right: the fraction of responses each critic values at exactly 0 or 1. These two panels are in tension — the judge with the widest separation is also the one most often pinned at an extreme — and Section 5.2 takes that up.

within them.

The question the two panels actually pose is therefore narrower than we framed it, and we have not answered it here: whether the gradient a critic supplies *between* probe points matters more to PPO than the separation it achieves at the end. That is decided by the PPO arms, not by this measurement, and we resist reading it off a correlation.

5.3 How fast does the sparse-probe bias vanish?

The mean error decays as $K^{-1.03}$, which is the Lipschitz rate of Proposition 2 essentially exactly, and *not* the K^{-2} that bounded curvature would give. The worst-case error behaves differently: it decays as $K^{-0.35}$, far more slowly, and is still 0.12 at $K = 16$.

We read the gap between the two as evidence that the value curve is Lipschitz but not smooth — that it contains localised near-discontinuities. That is what one should expect from the task rather than from the estimator: a response can commit to being bad at a particular token, by opening a self-promotional clause or by turning evasive, and the expected final quality drops abruptly there. No amount of uniform probing recovers a step.

Two things follow. First, K buys mean accuracy efficiently and worst-case accuracy barely, so the usual instinct to raise K until the critic “looks converged” spends calls on the wrong quantity. At $K = 6$, the setting we use, the mean error is 0.036 in probability units, which the calibration of Section 5.2 maps to roughly 0.08 in reward units on a scale of width 5. Second, and more usefully, it suggests that probes should not be placed uniformly: allocating them where the value curve moves fastest would attack precisely the error that K does not. We leave adaptive probe placement to future work, but note that the measurement here is what identifies it as the right thing to try.

Table 6: Bias of the sparsely-probed critic, measured against a densely probed value curve on $n = 57$ completions (every 2 tokens, 85 probes each). For each K the sparse estimate is rebuilt from K of the same probes, so the comparison isolates interpolation error from judge noise. The fitted decay is $K^{-0.84}$ for the mean error and $K^{-0.26}$ for the worst case; by Proposition 2 an exponent near -1 indicates a merely Lipschitz value curve and near -2 a curve with bounded curvature.

K probes	Segments	Mean $ \hat{V} - V^* $	Max $ \hat{V} - V^* $
2	1	0.1301	0.2875
3	2	0.0727	0.2566
4	3	0.0568	0.2454
6	5	0.0392	0.2183
8	7	0.0317	0.2029
12	11	0.0252	0.1917
16	15	0.0194	0.1730
24	23	0.0142	0.1454

Table 7: GRPO with the reward source as the only manipulated variable, at a matched budget of reward calls. Win-rate is against the SFT checkpoint, length-controlled, judged by a model unrelated to every training judge. Mean $\pm$ s.d. over three seeds.

Reward source	LC win-rate (%)	Raw (%)	GSM8K (%)	Len (tok)	Self-praise (%)	KL/tok
---------------	-----------------	---------	-----------	-----------	-----------------	--------

5.4 GRPO: varying the reward source

5.5 PPO: varying the critic

5.6 Reward hacking and over-optimisation

5.7 Where the rubric comes from

5.8 Cost

6 Discussion

What a typed interface buys, and what it does not. The advantages we can attribute to JEV are structural rather than mysterious: a rubric is a set of predeclared questions, which is its native input; answers arrive as numbers, so nothing is lost to parsing; and a judgment costs one prefill, so a dense multi-criterion reward is affordable at RL scale. None of this implies JEV is a *better* evaluator than a large LLM judge, and Section 5.1 measures that separately rather than assuming it. The interesting claim is narrower: for the specific shape of question RL needs answered, millions of times, the typed interface removes cost and failure modes that the generative interface imposes.

Why the critic role is the one that could not be filled before. An LLM judge could in principle be asked our prefix question too. What makes it impractical is arithmetic: a critic is queried at K positions per completion, so the query count is K times the reward count, and at LLM-judge latency and price that dominates the run. The typed model changes the constant factor by enough to change which algorithms are reachable. This is the sense in which the result is about a class of models and not about one vendor.

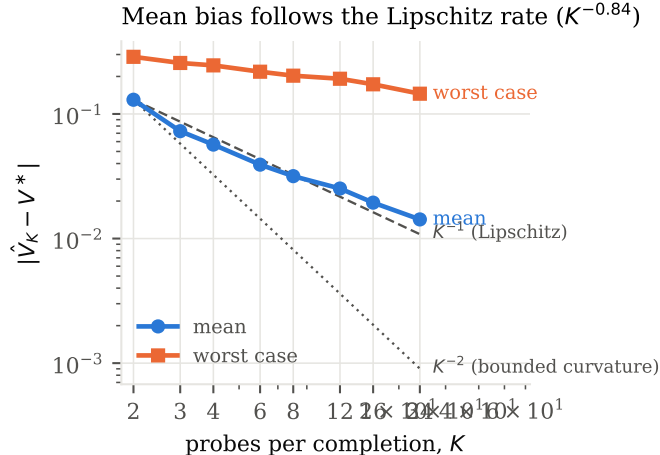


Figure 2: Interpolation error against the number of probes, measured against a densely probed value curve, with the two rates of Proposition 2 anchored at $K = 2$ for slope comparison. The mean error tracks the Lipschitz rate; the worst case does not, which is the signature of a value curve that steps rather than bends.

Table 8: PPO with the critic as the only manipulated variable: a learned value head versus JEV valuing token prefixes zero-shot. Both arms use the same reward source and the same PPO implementation.

Critic	LC win-rate (%)	Raw (%)	GSM8K (%)	Len (tok)	Self-praise (%)	KL/tok
--------	-----------------	---------	-----------	-----------	-----------------	--------

On self-promotion as an attack surface. The policy’s text is inside the judge’s state, which makes “write something the judge reads as evidence of quality” a gradient direction like any other. We both penalise it in the rubric and track it independently, and we report the cross-reward matrix so that a policy which moved towards its own judge is visible as a gap rather than as an improvement.

7 Limitations

JEV is a closed model with no published architecture or weights, so its calibration is an empirical property we measure rather than one we can reason about from first principles, and it may shift between versions; we pin `jev-1.13` and record the dated model string returned with every response. Our policy scale (0.5B) is set by the compute available and the ranking of reward sources may not transfer to substantially larger policies. The prefix critic is probed at K positions and interpolated, so the resulting advantages are piecewise approximations of a true token-level value function. Finally, using a hosted model’s outputs as a training signal is subject to that provider’s terms of service, which we checked before running.

8 Conclusion

[on completion of runs]

Table 9: Cross-reward matrix.

[pending: run not yet complete]

Table 10: Where the JEV question set comes from (STRUCTUREDGENERATION2JEV). All three arms are otherwise identical GRPO runs with JEV as the reward source. The *policy* arm lets the model being trained write its own grading criteria and is included as a reward-hacking probe.

Rubric source	LC win-rate (%)	Raw (%)	GSM8K (%)	Len (tok)	Self-praise (%)	KL/tok
---------------	-----------------	---------	-----------	-----------	-----------------	--------

References

- Yuntao Bai, Saurav Kadavath, Sandipan Kundu, Amanda Askell, Jackson Kernion, et al. Constitutional ai: Harmlessness from ai feedback. *arXiv preprint arXiv:2212.08073*, 2022.
- Ralph Allan Bradley and Milton E. Terry. Rank analysis of incomplete block designs: I. the method of paired comparisons. *Biometrika*, 39(3/4):324–345, 1952.
- Karl Cobbe, Vineet Kosaraju, Mohammad Bavarian, Mark Chen, Heewoo Jun, et al. Training verifiers to solve math word problems. *arXiv preprint arXiv:2110.14168*, 2021.
- Ganqu Cui, Lifan Yuan, Ning Ding, Guanming Yao, Wei Zhu, et al. Ultrafeedback: Boosting language models with high-quality feedback. *arXiv preprint arXiv:2310.01377*, 2023.
- Yann Dubois, Balázs Galambosi, Percy Liang, and Tatsunori B. Hashimoto. Length-controlled alpaca-eval: A simple way to debias automatic evaluators. *arXiv preprint arXiv:2404.04475*, 2024.
- Leo Gao, John Schulman, and Jacob Hilton. Scaling laws for reward model overoptimization. In *International Conference on Machine Learning (ICML)*, 2023.
- Harrison Lee, Samrat Phatale, Hassan Mansoor, Thomas Mesnard, Johan Ferret, et al. Rlaif vs. rlhf: Scaling reinforcement learning from human feedback with ai feedback. *arXiv preprint arXiv:2309.00267*, 2024.
- Long Ouyang, Jeffrey Wu, Xu Jiang, Diogo Almeida, Carroll Wainwright, et al. Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2022.
- Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2025.
- John Schulman, Philipp Moritz, Sergey Levine, Michael Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations (ICLR)*, 2016.
- John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.

Table 11: Measured cost of one training run per reward source. Local judges cost GPU-hours rather than dollars; API judges cost both. Latency is per reward call as observed by the trainer, including network round-trip.

Arm	Reward calls	USD	p50 (s)	p95 (s)	GPU-h
-----	--------------	-----	---------	---------	-------

Nisan Stiennon, Long Ouyang, Jeffrey Wu, Daniel M. Ziegler, Ryan Lowe, et al. Learning to summarize from human feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020.

TypeSafe AI. Jev: A system one model returning typed, calibrated decisions. <https://docs.typesafe.ai/api>, 2026. Model `jev-1.13`, accessed September 2026 via OpenRouter.

Leandro von Werra, Younes Belkada, Lewis Tunstall, Edward Beeching, Tristan Thrush, et al. TRL: Transformer reinforcement learning. <https://github.com/huggingface/trl>, 2020.

Weizhe Yuan, Richard Yuanzhe Pang, Kyunghyun Cho, Sainbayar Sukhbaatar, Jing Xu, and Jason Weston. Self-rewarding language models. *arXiv preprint arXiv:2401.10020*, 2024.

Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, et al. Judging llm-as-a-judge with mt-bench and chatbot arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023.